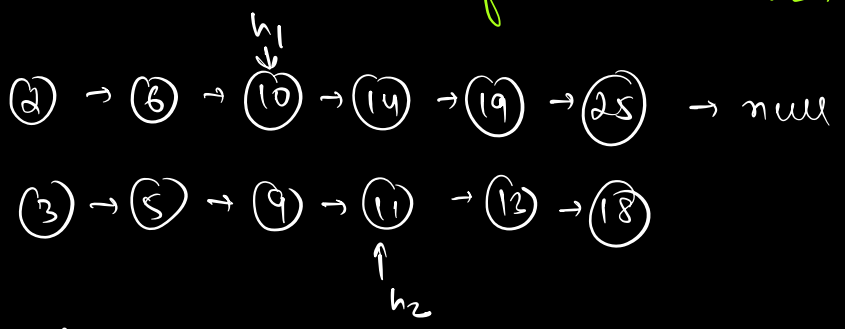
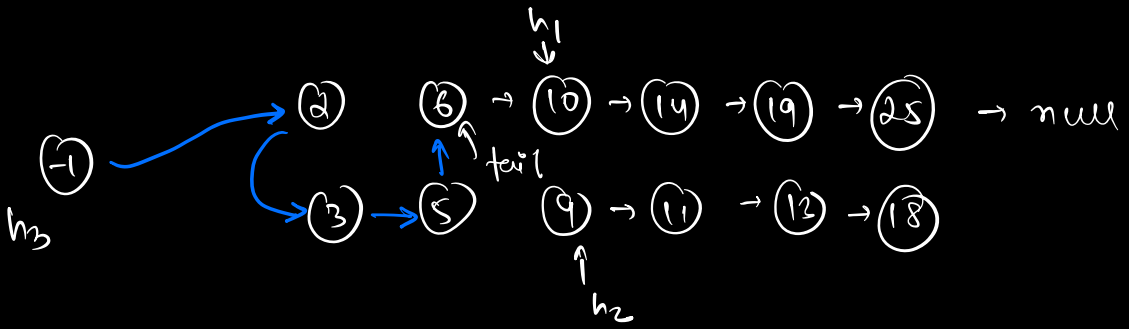
Q1. Merge 2 sorted LL into a final sorted LL.



* dummy Node

* dont use dummy Node

H1	H2
tail.next = h1	tail.next = h2
h1 = h1.next	h2 = h2.next
tail = tail.next	tail = tail.next



```
Node merge( h1 , h2 ) {
    Node h3 = null   tail = null

    [ Node h3 = new Node(-1);  ⟶ dummy node
      tail = h3 ]

    if( h1 == null ) return h2
    if( h2 == null ) return h1

    if ( H1.data <= H2.data) {
        h3 = h1
        h1 = h1.next
        tail = h3
    else
        h3 = h2
        h2 = h2.next
        tail = h3
    }
```

use if not using dummy node

```
while( h1 != null & h2 != null) {
    if( h1.data <= h2.data) {
        tail.next = h1
        h1 = h1.next
        tail = tail.next
    } else {
```

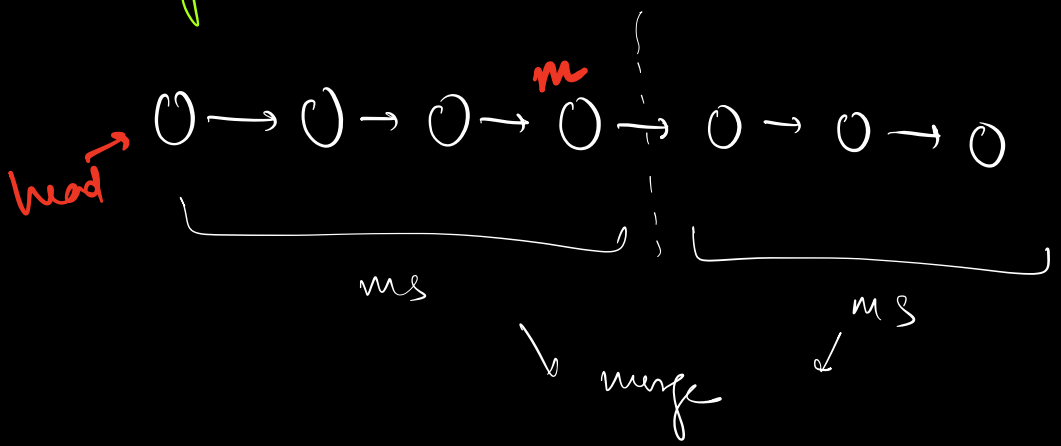
```
tail.next = h2
h2 = h2.next
tail = tail.next
}
return h3.next
```

h3 → dummy not used

h3.next → dummy used

TC => O(N+M)

Q2. Sort your LinkedList




```
Node mergesort(head){
if(head==null || head.next==null)
    return head;
Node m = middle(head);
Node h2 = m.next;
m.next = null;
head = mergesort(head)
h2 = mergesort(h2)
```

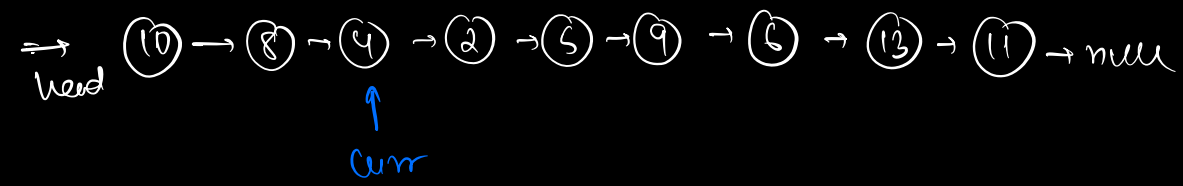
TC => O(NlogN)

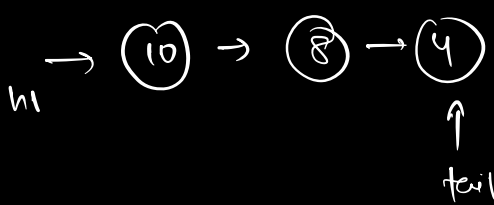
SC => O(1) + recursion stack

return merge(head, h2);

}

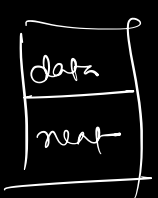
⇒ Clone a LL :





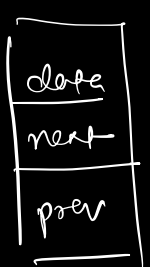
Doubly LL :

Singly LL →

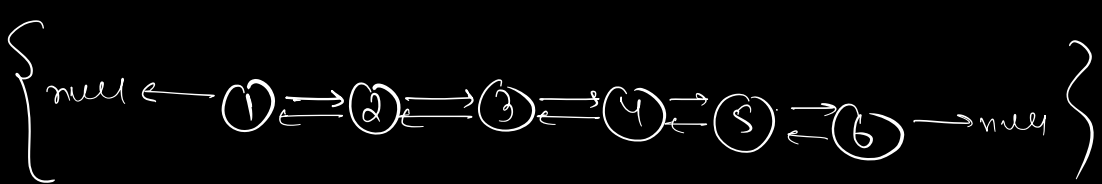


Node

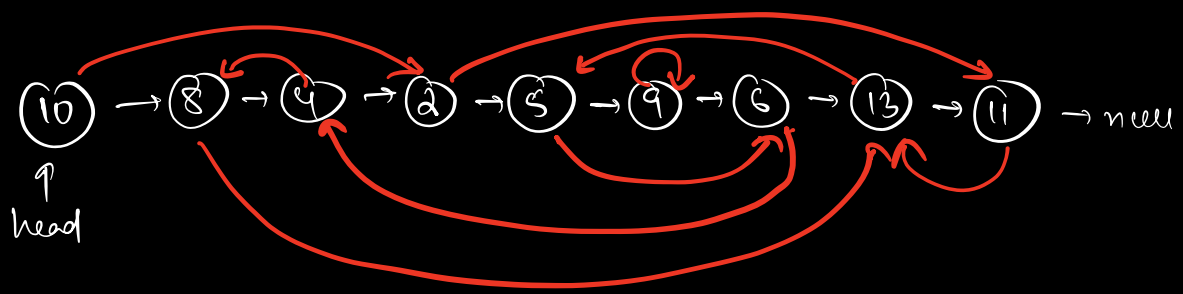
Node

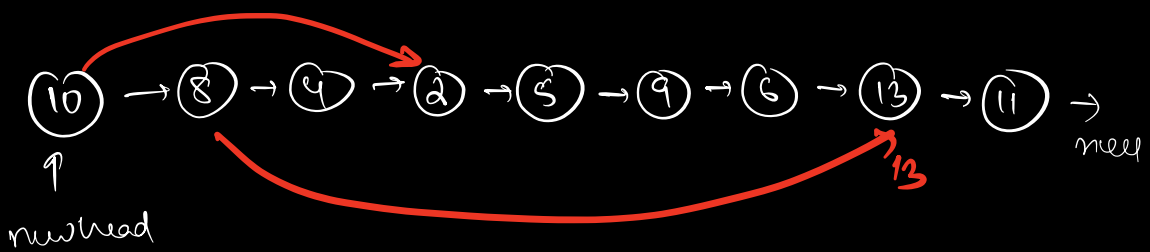


LL

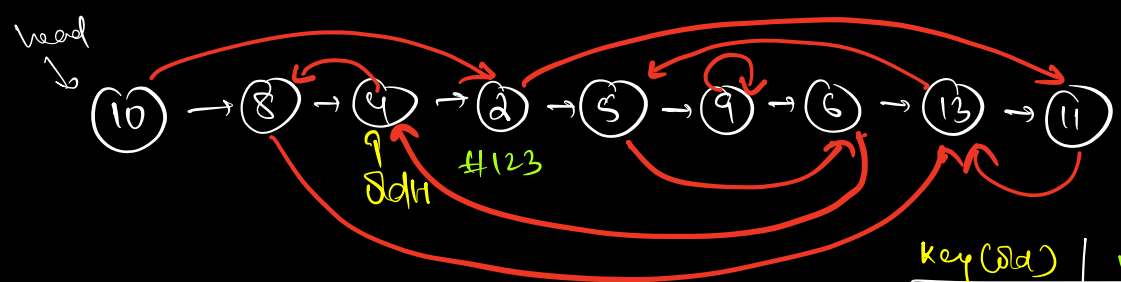


→ Clone a LL containing two ptr. → next & random

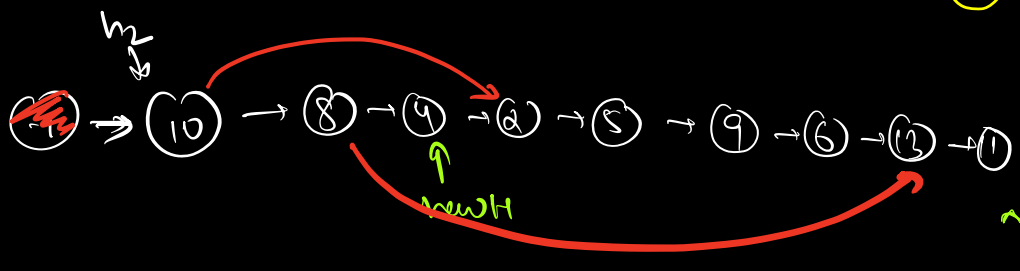




TC → $O(N^2)$ → for each node finding random ptr



key (old)	value (new)
10	10



new.random = x

`new.random = hm.get(old.random)`

Hashmap <Node, Node>

key (old)	value (new)
2(y) #123	2(x) #567

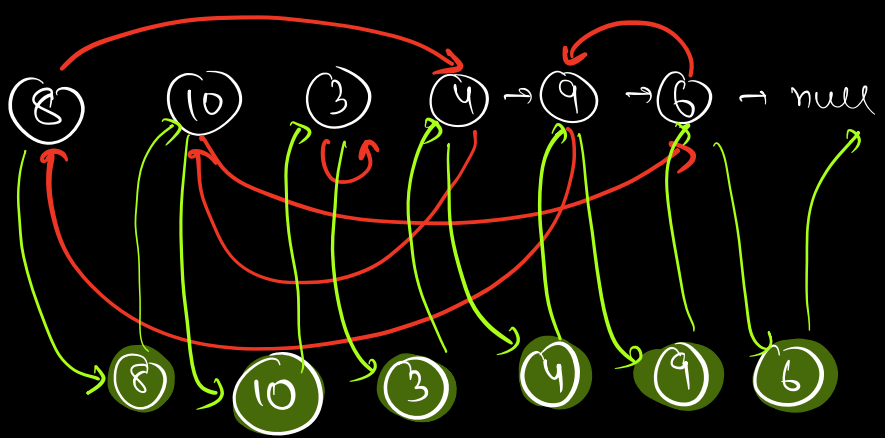
HM < Node, Node >

```
newH = new Node(-1) // dummy node
tail = newH
oldH = head
while( oldH != null ) {
        temp = new Node( oldH.data);
        tail.next = temp;
        tail = tail.next;
        HM.insert( oldH, tail);
        oldH = oldH.next;
}
    newH = newH.next // remove the dummy node
    oldH = head;
    h2 = newH;
    while ( oldH != null) {
        x = HM.get( old.random)
        newH.random = x;
```

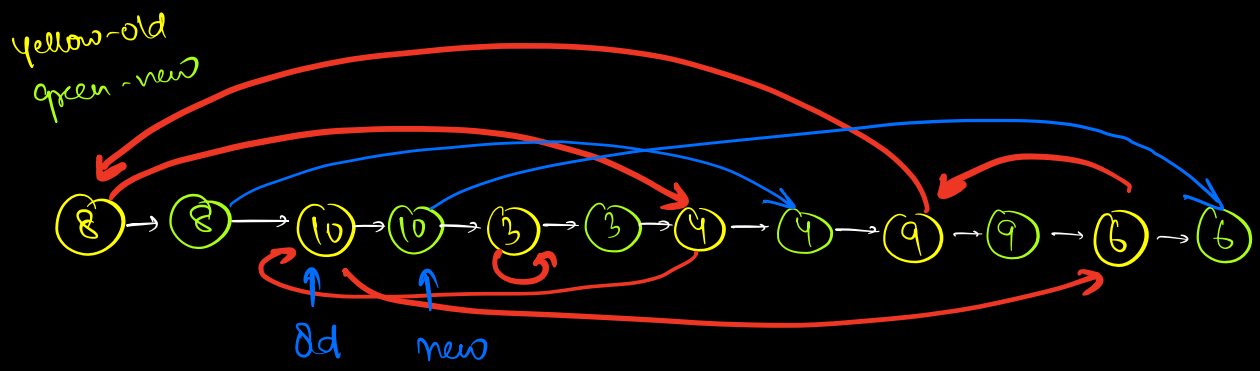
```
oddH = oddH.next;
newH = newH.next;
}
```

TC → O(N)
SC → O(N)

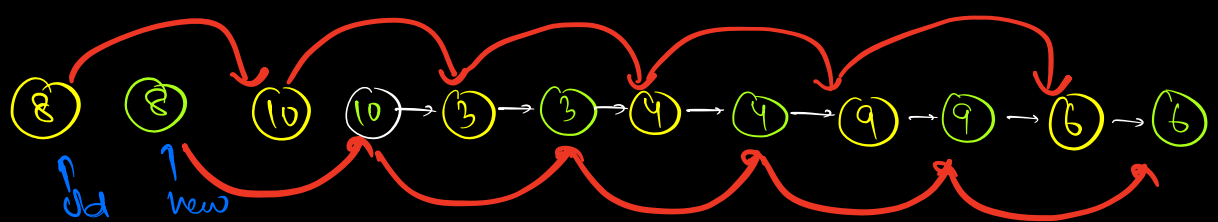
→ no extra space :



~~curr.next~~ curr.next.next

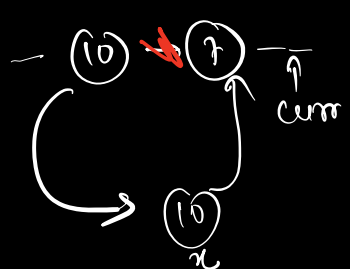



```
new.random = old.random.next
old = old.next.next
new = new.next.next
```



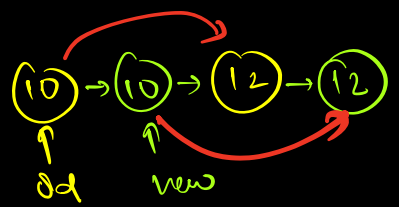

```
old.next = old.next.next;
new.next = new.next.next;
```

Step 1 → Create new nodes b/w old nodes :-



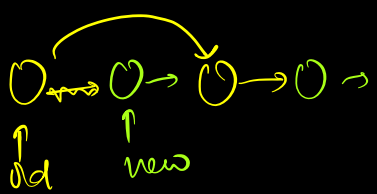

```
curr = head
while(curr != null) {
    x = new Node(curr.data);
    x.next = curr.next
    curr.next = x
} curr = curr.next.next
```

Step2 : update random ptrs of new nodes :-




```
Old = head;
new = head.next;
while( Old != null ) {
    new.random = Old.random.next;
    Old = Old.next.next
    new = new.next.next
}
```

Step-3 → split / separate Old & new nodes




```
Node ans = new;
while( Old != null ) {
    Old.next = Old.next.next
    new.next = new.next.next
    Old = Old.next
    new = new.next
}
return ans;
```

TC = O(3N) ≈ O(N)

SC = O(1)

⇒ **LRU cache :-**

↓

least recently used

7	3	9	2	6	10	14	2	10	15	8	14
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
x	x	x	x	x	x	x	✓	✓	x	x	✓

capacity = 5

HIT → got it from cache

MISS → did not get from cache

7 3 9 2 6

↓ -7 / +10

3 9 2 6 10

↓ -3 / +14

9 2 6 10 14

9 6 10 14 2

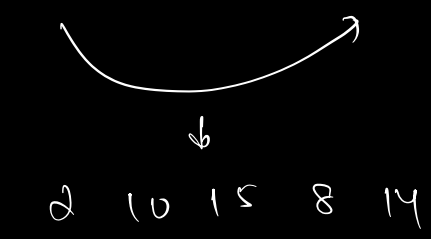
9 6 14 2 10

↓ -9 / +15

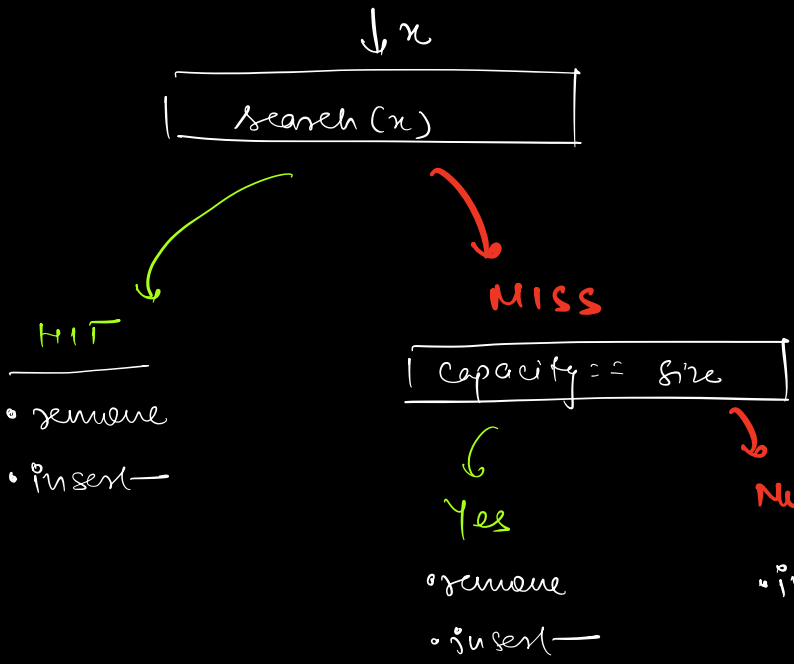
6 14 2 10 15

↓ -6 / +8

14 2 10 15 8

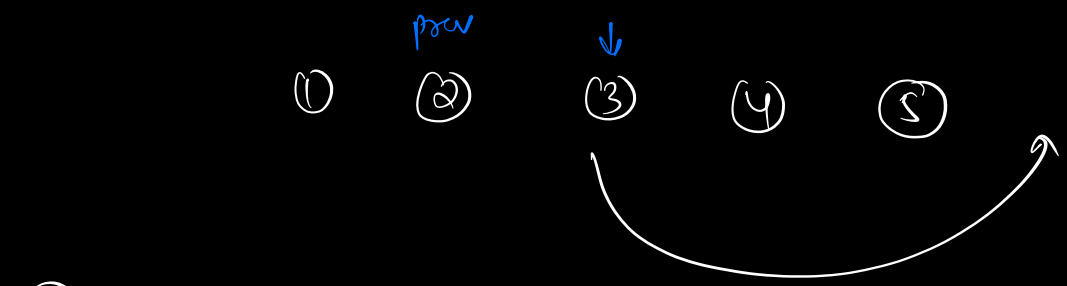


- search
- remove
- insert

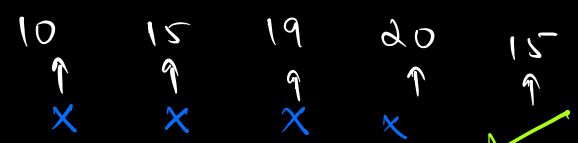


	Array	LL	LL + HM	DLL + HM
Search	O(N)	O(N)	O(1)	O(1)
insert	O(1)	O(1)	O(1)	O(1)
remove	O(N)	O(1)	O(1) prev	O(1)

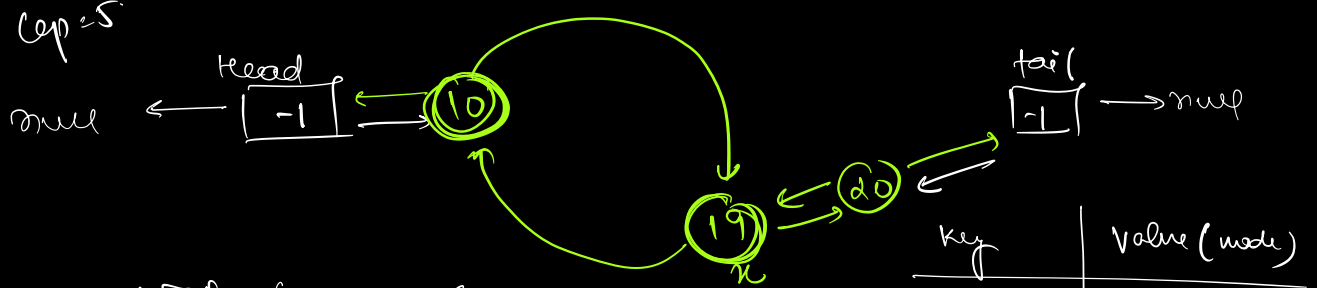
Search already brought ph. to node



10 15 19 20 15 18 23 20 19 17 17



cap = 5



Key	Value (node)
10	(10)
15	(15)
19	(19)
20	(20)

```
addToEnd(Node x) {
    x.next = tail
    x.prev = tail.prev
    tail.prev = x
    x.prev.next = x;
```

```
remove(Node x) {
    x.prev.next = x.next;
    x.next.prev = x.prev;
}
```